

CODE OPTIMIZATION

4

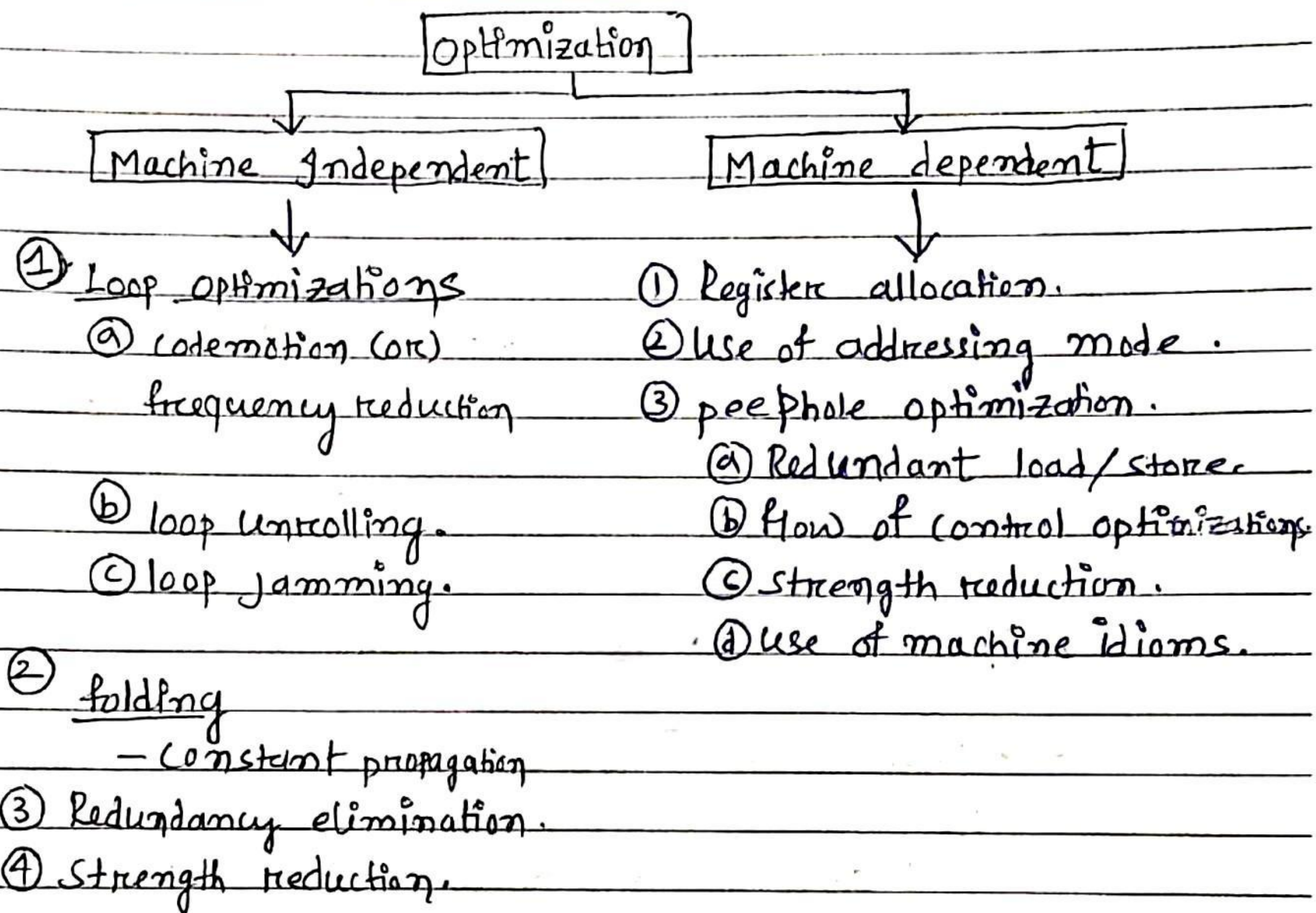
classmate

Date

Page

92

Code Optimization Introduction :



M/c Independent =

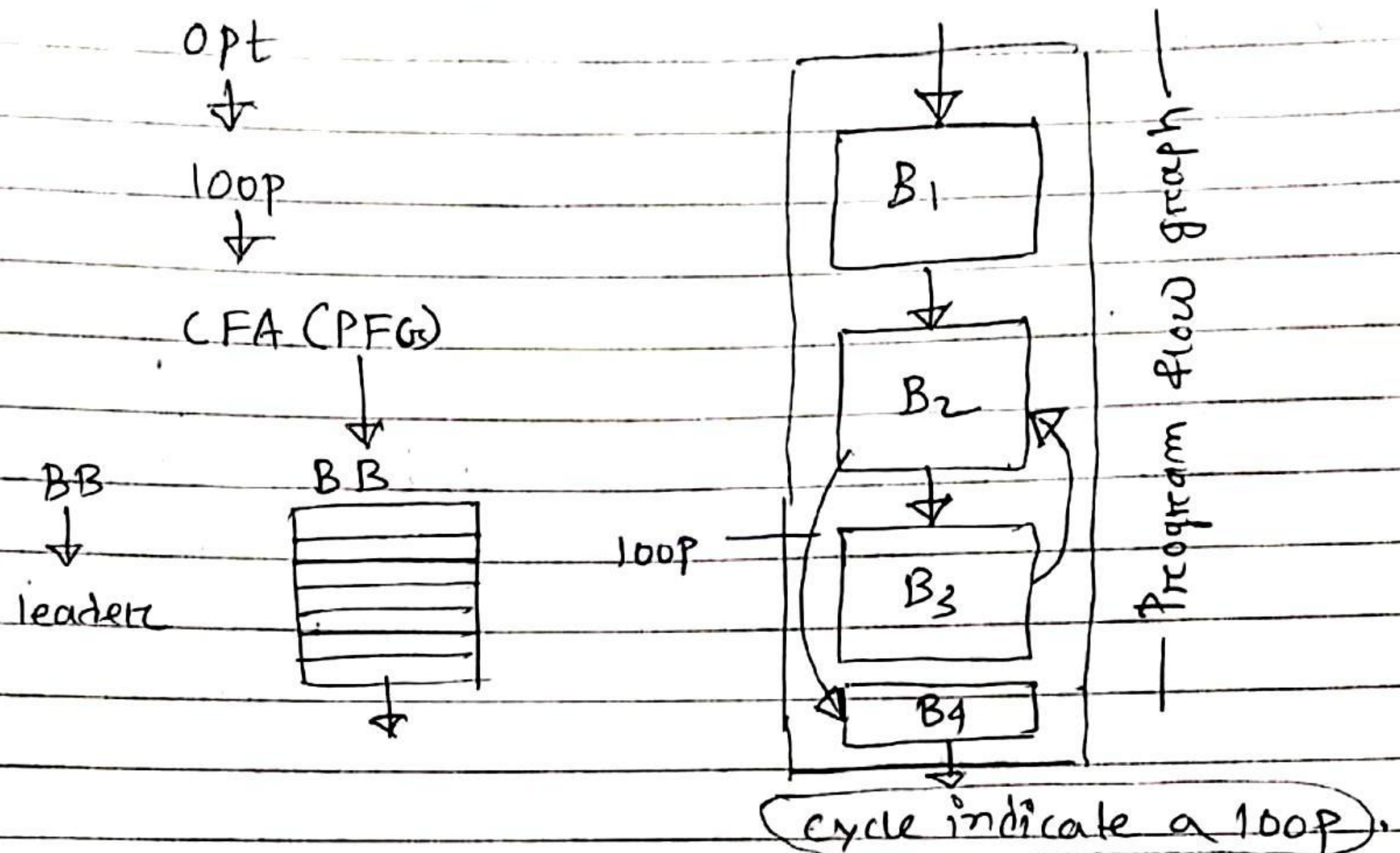
① Loop Optimizations =

→ To apply loop optimizations, we must first detect loops.

→ For detecting loops we use control flow analysis (CFA) using ~~the~~ program flow graph.

→ To find PFC, we need to find basic blocks.

A Basic block is a sequence of 3-address code statements where control enters at the beginning and leaves only at the end without any jumps or halts.



• Algorithm to find the Basic Block:

→ In order to find the basic blocks, we need to find the leaders in program. Then a basic block will start from one leader to the next leader but not including next leader.

• Identifying leaders in a basic block:

- ① A statement is a leader.
- ② statement that is target of conditionals or unconditional statement is leader.
- ③ Statement that follows immediately a conditional or unconditional statement is a leader.

Example = (find out leaders)

Program -

```
fact(n)
{
  int f = 1;
  for (i = 2; i <= n; i++)
    f = f * i;
  return f;
}
```

3 add code -

* 1) $f = 1$ → leader, (not including other leader) - we make block
 2) $i = 2$ B_1

leader ← * 3) if ($i > n$) goto (9) B_2

* 4) $t_1 = f * i$ → leader
 5) $f = t_1$
 6) $t_2 = i + 1$ B_3
 7) $i = t_2$
 8) goto (3).

* 9) End of calling program (Return) → leader B_4

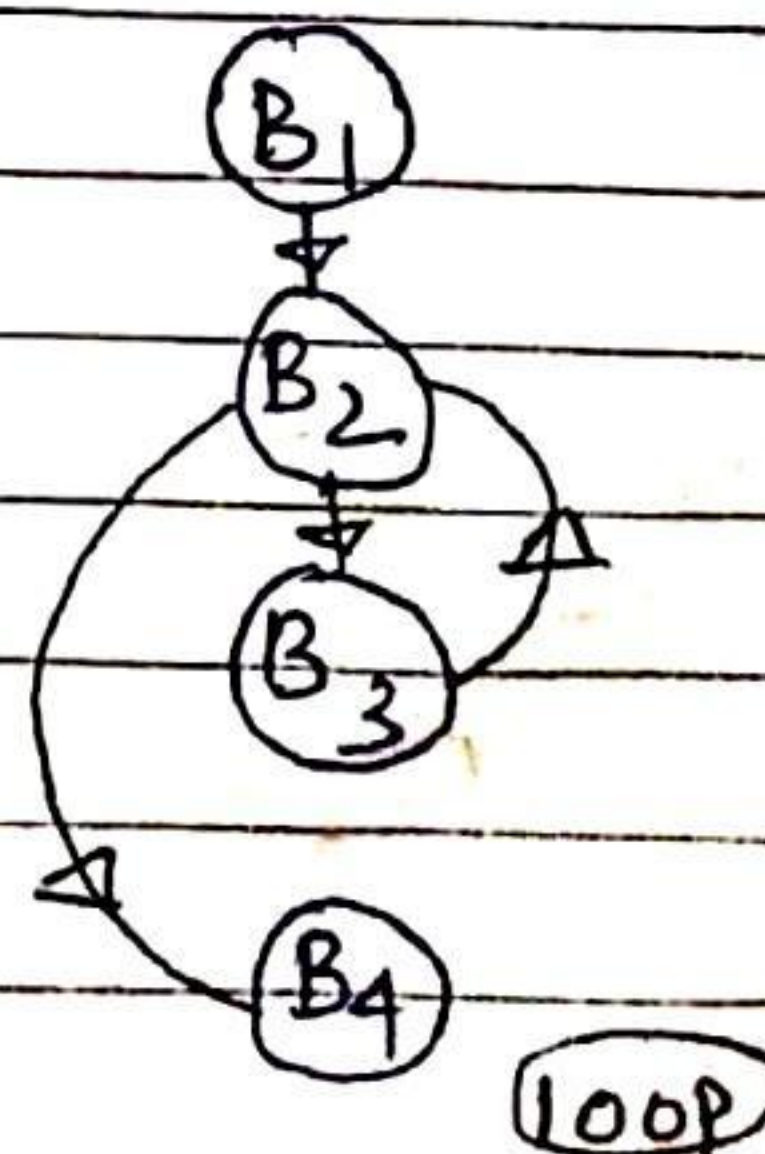
→ using Algo we find '4' leaders and '4' block.

→ no. of Basic Block depends on number of leaders.

→ with 'n' leaders we get 'n' Block.

Program Flow graph -

apply-CFA



B_2 and B_3 falling loop

• Type of loop optimization $\Rightarrow$

① Frequency reduction:

Moving the code from high frequency region to low frequency region is called code motion.

Ex:

```
while (i < 5000)
{
    A = sin(x) / cos(x) * i;
    i++;
}
```



```
t = sin(x) / cos(x);
while (i < 5000)
    A = t * i;
```

② Loop unrolling: effectively reduce the number of computation:

```
while (i < 10)
{
    x[i] = 0;
    i++;
}
```



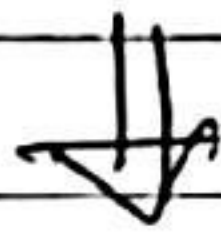
```
while (i < 10)
{
    x[i] = 0;
    i++;
    x[i] = 0;
    i++;
}
```

[0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

③ loop jamming: combining the bodies of two loops.

```
for (i=0; i<10; i++)
  for (j=0; j<10; j++)
```

```
    u[i,j] = 0;
for (i=0; i<10; i++)
    u[i,i] = 0;
```



```
for (i=0; i<10; i++)
{
  for (j=0; j<10; j++)
  {
    u[i,j] = 0;
    u[i,i] = 0;
  }
}
```

② folding =

Replacing an expression that can be computed at compile time by its values.

Ex: $2+3+C+B = 5+C+B$

③ Redundancy Elimination = (DAG)

$$A = B + C$$

$$D = 2 + B + 3 + C$$

$$D = 2 + 3 + A$$

$$= 5 + A$$

- ④ Strength reduction = Replacing a costly operation cheaper one.

Ex: $B = A * 2$
 $B = A \ll 1.$

- ⑤ Algebraic simplification =

$A = A + 0$ } eliminate
 $x = x * 1$ } such statements.

M/c dependent opt =

co-disc

- ① Register allocation

local allocation

global allocation.

co-disc

- ② Use of addressing modes.

- ③ peephole optimization =

(a) Redundant load and store elimination =

$x = y + z$ $\left\{ \begin{array}{l} \text{MOV } y, R_0 \\ \text{add } z, R_0 \\ \text{MOV } R_0, x \end{array} \right.$

$a = b + c$

$d = a + e$

~~MOV~~ b, R₀

add c, R₀

MOV R₀, a

MOV a, R₀

add e, R₀

MOV R₀, d.

x

(b) Flow of control optimization =

Avoid
Jumps on
Jumps

L1: ~~Jump~~ L4

⋮

L2: ~~Jump~~ L3

⋮

L3: ~~Jump~~ L4

eliminate
dead code

define 0

if(x)

{

↑

dead code

}

(d) use of m/c idioms =

i = i + 1

mov R0, i

add R0, 1

mov i, R0

⇒ inc i